# Contents

**1 ITP-Lab Question** 2
1.1 Problem-Statement . . . 2
1.2 Implementation . . . 2
1.2.1 Input Images . . . 2
1.2.2 Grayscaled Images . . . 4
1.2.3 Method-1: Fixed Thresholding . . . 5
1.2.4 Method-2: Average Thresholding . . . 7
1.2.5 Comparative Analysis and Conclusion . . . 9

**Appendix A: Image-Processing: Theory** 10
A.1 Introduction . . . 10
A.2 RGB Image Representation . . . 10
A.2.1 JPEG Representation . . . 10
A.2.2 Per-Pixel Storage . . . 11
A.3 Colour Conversion . . . 12
A.3.1 RGB to Luminance and Chrominance . . . 12
A.3.2 Chrominance Components . . . 12
A.3.3 Luminance Weighting . . . 12
A.3.4 Information Reduction . . . 12
A.4 RGB to Grayscale Conversion . . . 13
A.4.1 Grayscale Intensity . . . 13
A.4.2 Black and White . . . 13
A.5 Grayscale and Black-and-White Conversion Algorithm . . . 13
A.5.1 RGB to Grayscale Algorithm . . . 13
A.5.2 Mapping to the Output Image . . . 14
A.6 Thresholding Methods and Analysis . . . 14
A.6.1 Global Mean Thresholding . . . 14
A.6.2 Static Reference Thresholding . . . 14

**References** 15

ation for the RGB image-processing project.

## 1.1 Problem-Statement

Image processing is the computational manipulation and analysis of digital images. A digital image is represented by pixels containing RGB (Red, Green, Blue) values, arranged in a two-dimensional array for a 2D image.

The luminance intensity $Y$ of each pixel is calculated from its RGB components using the relation given in Appendix (A.3.1.1):

$$Y = 0.299R + 0.587G + 0.114B \tag{1.1.1}$$

The resulting luminance values form a grayscale image. Given a threshold value $T$, convert this grayscale image into a binary black-and-white image using the thresholding relation given in Appendix (A.4.2.1):

$$B(x, y) = \begin{cases} 0, & Y(x, y) \leq T, \\ 255, & Y(x, y) > T. \end{cases} \tag{1.1.2}$$

Store the resulting binary image in a separate array without modifying the original grayscale image.

## 1.2 Implementation

The implementation is presented in four stages: the original input images, their grayscale versions, fixed-threshold outputs, and average-threshold outputs. The same order of subjects is retained throughout to make the results directly comparable.

### 1.2.1 Input Images

The original images used in this project are shown below. They are retained as the reference for comparing the grayscale conversion and the two thresholding methods.

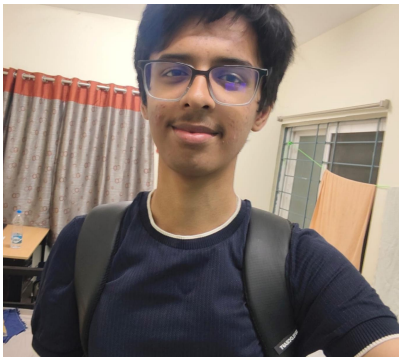


Fig. 1.2.1.1: Arjun

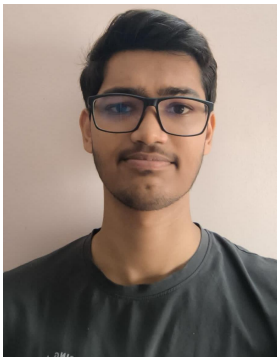


Fig. 1.2.1.2: Arya



Fig. 1.2.1.3: Mudit

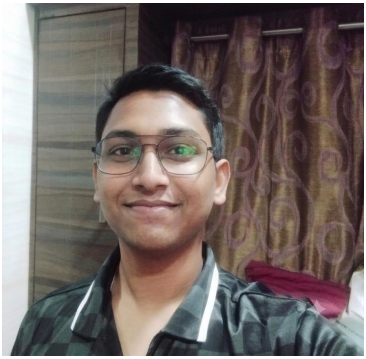


Fig. 1.2.1.4: Visweshwar

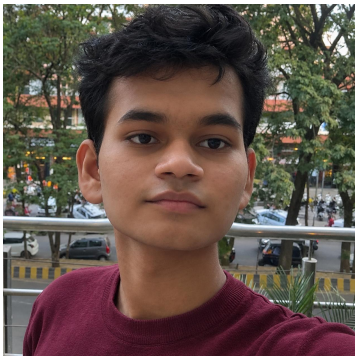


Fig. 1.2.1.5: Saksham

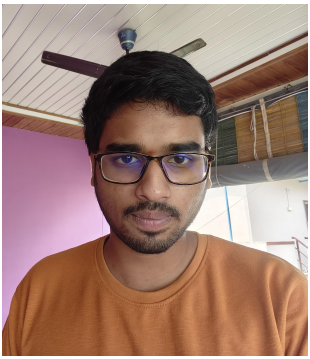


Fig. 1.2.1.6: Pranay



Fig. 1.2.1.7: Nethra



Fig. 1.2.1.8: Sara

### 1.2.2 Grayscaled Images

Each input image is converted into a grayscale image by reducing the three RGB components to one luminance value, $Y(x, y)$, using the luminance equation defined in the Appendix. The resulting intensity image preserves brightness information while removing colour information.

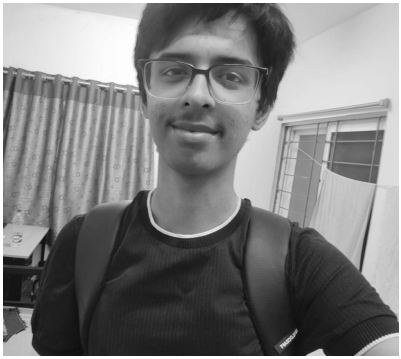


Fig. 1.2.2.1: Arjun



Fig. 1.2.2.2: Arya



Fig. 1.2.2.3: Mudit



Fig. 1.2.2.4: Visweshwar

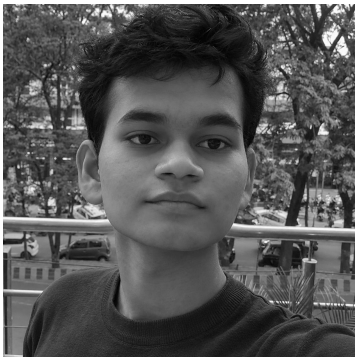


Fig. 1.2.2.5: Saksham



Fig. 1.2.2.6: Pranay

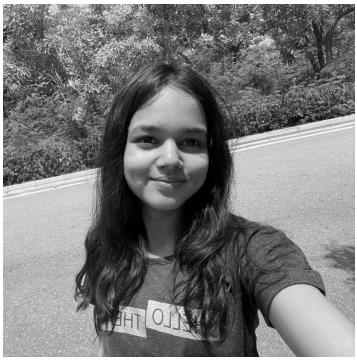


Fig. 1.2.2.7: Nethra



Fig. 1.2.2.8: Sara

### 1.2.3 Method-1: Fixed Thresholding

In fixed thresholding, a constant threshold $T$ is selected before processing the images. For each pixel, the luminance $Y(x, y)$—calculated using the equation defined in the Appendix—is compared with $T$. If $Y(x, y) \leq T$, the output pixel is set to 0 (black); otherwise, it is set to 255 (white).

A limitation of fixed thresholding is that one threshold may not classify foreground and background effectively across images with different brightness and contrast. Consequently, large regions can become the same colour in the binary output and useful details may be lost. This limitation motivates the image-dependent average-thresholding method described next.



Fig. 1.2.3.1: Arjun (Fixed Threshold)



Fig. 1.2.3.2: Arya (Fixed Threshold)



Fig. 1.2.3.3: Mudit (Fixed Threshold)



Fig. 1.2.3.4: Visweshwar (Fixed Threshold)



Fig. 1.2.3.5: Saksham (Fixed Threshold)



Fig. 1.2.3.6: Pranay (Fixed Threshold)

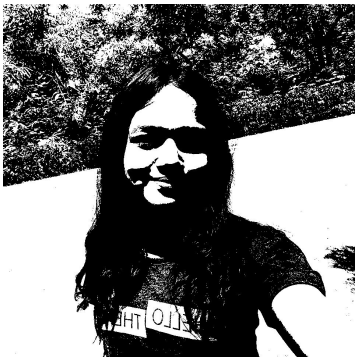


Fig. 1.2.3.7: Nethra (Fixed Threshold)



Fig. 1.2.3.8: Sara (Fixed Threshold)

The code to implement this is:

codes/auto/static.py

### 1.2.4 Method-2: Average Thresholding

In average thresholding, the threshold is calculated from the grayscale intensities of the current image instead of being fixed in advance. For an image with $M \times N$ pixels, the average threshold is

$$T_{\text{avg}} = \frac{1}{MN} \sum_{x=1}^{M} \sum_{y=1}^{N} Y(x, y). \tag{1.2.3}$$

Each pixel is classified by comparing its luminance with $T_{\text{avg}}$: pixels with $Y(x, y) \leq T_{\text{avg}}$ become black, while pixels with $Y(x, y) > T_{\text{avg}}$ become white. The advantage is relativity: the threshold responds to the overall brightness distribution of each input image rather than applying the same threshold to every image. Since this is a single image-wide average, it does not necessarily compensate for uneven lighting at different locations within the same image.



Fig. 1.2.4.1: Arjun (Dynamic Threshold)



Fig. 1.2.4.2: Arya (Dynamic Threshold)



Fig. 1.2.4.3: Mudit (Dynamic Threshold)



Fig. 1.2.4.4: Visweshwar (Dynamic Threshold)



Fig. 1.2.4.5: Saksham (Dynamic Threshold)



Fig. 1.2.4.6: Pranay (Dynamic Threshold)

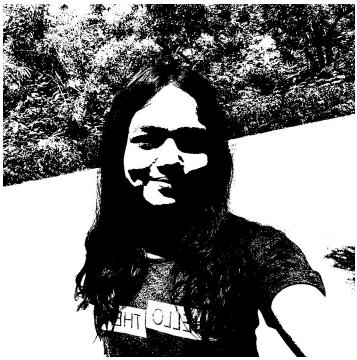


Fig. 1.2.4.7: Nethra (Dynamic Threshold)



Fig. 1.2.4.8: Sara (Dynamic Threshold)

The code to implement this is:

codes/auto/dynamic.py

### 1.2.5 Comparative Analysis and Conclusion

The fixed-threshold and average-threshold outputs are compared separately for each input image. Fixed thresholding uses the constant reference $T = 128$, whereas average thresholding calculates a global threshold from the mean grayscale intensity of the current image. The comparison focuses on the visibility of relevant features, separation of foreground and background, preservation of useful details, and overall interpretability. Because these properties must be judged from the actual output images, a winner is assigned independently for each case rather than assuming that either method is always superior.

*1.2.5a Summary of results:* The winning method for each input image is recorded in the table below. The classification is based on the comparison of the corresponding fixed (static) and mean-based (dynamic) thresholding outputs.

| **Static Wins** | **Dynamic Wins** | **Tie** |
|---|---|---|
| Mudit<br>Sara | Visweshwar<br>Saksham | Arjun<br>Arya<br>Nethra<br>Pranay |

*1.2.5b Conclusion:* The comparison is expected to show that thresholding performance depends on the image being processed. A fixed threshold provides a consistent reference and can work well when the subject and background are separated near that intensity. Mean-based thresholding adapts its reference to the overall grayscale intensity distribution, but because it uses one global mean, it may still perform poorly when illumination or contrast varies within an image. Neither method is guaranteed to produce the best result for every input.

Therefore, the preferred thresholding method must be selected on a case-by-case basis according to the image's characteristics and the purpose of the output. The winner for one image need not be the winner for another; the final selection should be justified by the observed detail preservation and foreground–background separation, rather than by a universal ranking of the methods.

# APPENDIX A
# IMAGE-PROCESSING: THEORY

## A.1 Introduction

An RGB image represents colour using three components: Red, Green, and Blue. A pixel can therefore be represented by the tuple

$$P_{ij} = (R_{ij}, G_{ij}, B_{ij}) \tag{A.1.1}$$

For a conventional 8-bit RGB image, each component is represented using 8 bits. Hence, one pixel requires

$$8 + 8 + 8 = 24 \tag{A.1.2}$$

bits of colour information.

For an image containing $M \times N$ pixels, the image may be viewed as three matrices,

$$R \in \mathbb{R}^{M \times N} \tag{A.1.3}$$

$$G \in \mathbb{R}^{M \times N} \tag{A.1.4}$$

$$B \in \mathbb{R}^{M \times N} \tag{A.1.5}$$

The conversion to black and white is therefore a reduction from three colour values per pixel to one intensity value per pixel.

## A.2 RGB Image Representation

A raw RGB bitmap stores the colour components of the pixels directly. For a $2 \times 2$ group of pixels,

$$\begin{bmatrix} (R_{11}, G_{11}, B_{11}) & (R_{12}, G_{12}, B_{12}) \\ (R_{21}, G_{21}, B_{21}) & (R_{22}, G_{22}, B_{22}) \end{bmatrix} \tag{A.2.1}$$

### A.2.1 JPEG Representation

JPEG uses a transformed and compressed representation rather than storing every RGB tuple independently. The JPEG standard defines processes for converting source image data to compressed image data.

The common colour information is calculated, and the formulae used for this conversion are listed in the next section.

### A.2.2 Per-Pixel Storage

For an image containing $N$ pixels, an 8-bit RGB representation requires three separate 8-bit colour components for every pixel. Hence, the raw colour information requires

$$3 \times 8N = 24N \tag{A.2.2.1}$$

bits.

If the image is converted directly from RGB to grayscale, only the luminance value $Y$ is retained for each pixel. With an 8-bit grayscale representation, this requires

$$1 \times 8N = 8N \tag{A.2.2.2}$$

bits.

Thus, the storage changes from $24N$ bits for RGB to $8N$ bits for grayscale. The storage ratio is therefore

$$\frac{8N}{24N} = \frac{1}{3} \tag{A.2.2.3}$$

so the grayscale representation requires one-third of the raw RGB storage, corresponding to a reduction of two-thirds.

When the image is represented using $Y$, $Cb$, and $Cr$, the three components are still separate components. Each component is normally represented using 8-bit samples; the reduction in storage can instead be obtained by reducing the spatial sampling of the chrominance components. In particular, with 4:2:0 chroma subsampling, the $Y$ component retains $N$ samples, while $Cb$ and $Cr$ each contain $N/4$ samples.

Therefore, before the additional JPEG transform and coding stages, the sample storage can be written as

$$8N + 8\frac{N}{4} + 8\frac{N}{4} = 12N \tag{A.2.2.4}$$

bits.

Thus, the factor of four applies to the *number of chrominance samples*, not to the number of bits in an individual $Cb$ or $Cr$ sample. The $Y$ component is retained at full spatial resolution because it carries the intensity information for every pixel.

For example, in a $2 \times 2$ group containing four $Y$ samples, 4:2:0 subsampling uses one $Cb$ sample and one $Cr$ sample for the group. The $Cb$ and $Cr$ samples are therefore stored separately, but they are shared spatially by the four corresponding pixels.

The comparison above describes sample storage before the main JPEG compression stages. The actual size of a JPEG file is different because JPEG additionally uses transformation, quantisation, and entropy coding as part of its coding process.

## A.3 Colour Conversion

### A.3.1 RGB to Luminance and Chrominance

The RGB components may be transformed into three separate components, luminance $Y$ and chrominance $Cb$ and $Cr$:

$$Y = 0.299R + 0.587G + 0.114B \tag{A.3.1.1}$$

$$Cb = -0.168736R - 0.331264G + 0.5B + 128 \tag{A.3.1.2}$$

$$Cr = 0.5R - 0.418688G - 0.081312B + 128 \tag{A.3.1.3}$$

Here, $Y$ represents luminance, while $Cb$ and $Cr$ represent chrominance information. The $Cb$ and $Cr$ components carry colour-difference information associated with blue and red, respectively. They are separate 8-bit components in the usual 8-bit representation; they are not single-bit colour indicators.

### A.3.2 Chrominance Components

The $Cb$ component describes the blue-related colour difference and $Cr$ describes the red-related colour difference. These components retain colour information that is not required when only grayscale intensity is required.

When chroma subsampling is used, the $Cb$ and $Cr$ components can be stored at a lower spatial sampling rate than $Y$. For example, in 4:2:0 subsampling, each $2 \times 2$ group of pixels retains four $Y$ samples but uses one $Cb$ sample and one $Cr$ sample. Thus, the chrominance components remain separate components, while fewer chrominance samples are stored.

### A.3.3 Luminance Weighting

The coefficients in the luminance equation are different because the human visual system does not perceive the three RGB components with equal sensitivity:

$$0.587 > 0.299 > 0.114 \tag{A.3.3.1}$$

The green component consequently has the largest contribution to luminance, followed by red and then blue.

### A.3.4 Information Reduction

For the purpose of grayscale conversion, the colour pixel $(R, G, B)$ is reduced to the single luminance value $Y$. The $Cb$ and $Cr$ components carry the colour-difference information and are therefore not required for the final grayscale intensity image.

## A.4 RGB to Grayscale Conversion

### A.4.1 Grayscale Intensity

A grayscale image represents every pixel using a single intensity value:

$$Y = 0.299R + 0.587G + 0.114B \tag{A.4.1.1}$$

A small value of $Y$ represents a dark pixel, while a large value represents a bright pixel.

### A.4.2 Black and White

A grayscale image contains many intensity levels between black and white. A black-and-white image instead contains two classes.

Let $T$ denote the selected threshold:

$$B(i,j) = \begin{cases} 0, & Y(i,j) \leq T, \\ 255, & Y(i,j) > T. \end{cases} \tag{A.4.2.1}$$

Thus the grayscale intensity is mapped to one of two output values.

## A.5 Grayscale and Black-and-White Conversion Algorithm

### A.5.1 RGB to Grayscale Algorithm

The conversion procedure is carried out pixel by pixel.

- Read the RGB input image.
- Obtain $R$, $G$, and $B$ for each pixel.
- Calculate the luminance $Y$.
- Store $Y$ as the grayscale intensity.
- Select the threshold $T$.
- Compare every grayscale value with $T$.
- Assign black or white according to the threshold condition.
- Save the resulting image.

The complete transformation is

$$RGB \rightarrow Y \rightarrow \text{Threshold} \rightarrow \text{Black/White} \tag{A.5.1.1}$$

### A.5.2 Mapping to the Output Image

The calculated intensity matrix is passed to the image-writing stage. When JPEG is selected, the image-writing stage performs the corresponding JPEG encoding and compression.

## A.6 Thresholding Methods and Analysis

### A.6.1 Global Mean Thresholding

The first dynamic method calculates one reference intensity from the complete grayscale image. If the grayscale matrix is denoted by $Y$, the reference is the mean intensity of all pixels:

$$\mu = \frac{1}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} Y(i,j) \tag{A.6.1.1}$$

Every pixel is then compared with this single image-wide reference. The corresponding binary decision is

$$B(i,j) = \begin{cases} 255, & Y(i,j) > \mu, \\ 0, & Y(i,j) \leq \mu. \end{cases} \tag{A.6.1.2}$$

This method is adaptive in the sense that the reference changes with the input image, but the reference itself is global. It therefore does not change from one location to another within the same image.

### A.6.2 Static Reference Thresholding

For comparison, a fixed reference of $128$ is used in the static thresholding method. A grayscale pixel is assigned to white when its intensity is at least $128$ and to black otherwise:

$$B(i,j) = \begin{cases} 255, & Y(i,j) > 128, \\ 0, & Y(i,j) \leq 128. \end{cases} \tag{A.6.2.1}$$

The purpose of this fixed reference is to provide a simple baseline against which the dynamically calculated references can be compared.

# References


[1] International Telecommunication Union, *Information Technology—Digital Compression and Coding of Continuous-Tone Still Images: JPEG File Interchange Format (JFIF)*, ITU-T Recommendation T.871, 2011. Available: https://www.itu.int/rec/T-REC-T.871

[2] International Telecommunication Union, *Information Technology—Digital Compression and Coding of Continuous-Tone Still Images—Requirements and Guidelines*, ITU-T Recommendation T.81, 1992. Available: https://www.itu.int/rec/T-REC-T.81

[3] International Telecommunication Union, *Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios*, ITU-R Recommendation BT.601. Available: https://www.itu.int/rec/R-REC-BT.601

[4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Hoboken, NJ, USA: Pearson, 2018.

[5] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, Jan. 1979, doi: 10.1109/TSMC.1979.4310076.

[6] S. van der Walt *et al.*, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, article e453, 2014, doi: 10.7717/peerj.453. Available: https://doi.org/10.7717/peerj.453

[7] scikit-image contributors, "Thresholding," *scikit-image Documentation*. Available: https://scikit-image.org/docs/stable/auto_examples/applications/plot_thresholding.html

[8] Pillow contributors, "Image module," *Pillow Documentation*. Available: https://pillow.readthedocs.io/en/stable/reference/Image.html